

Generative Classifiers & Naïve Bayes

Robin Jia

USC CSCI 467, Fall 2026

September 16, 2026

Review: Supervised learning methods (so far)

	Linear Regression	Logistic Regression	Softmax Regression
Task	<u>Regression</u> y is a real number	<u>Binary Classification</u> $y \in \{+1, -1\}$	<u>Multiclass classification</u> $y \in \{1, 2, \dots, C\}$

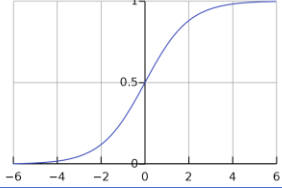
Review: Supervised learning methods (so far)

	Linear Regression	Logistic Regression	Softmax Regression
Task	<u>Regression</u> y is a real number	<u>Binary Classification</u> $y \in \{+1, -1\}$	<u>Multiclass classification</u> $y \in \{1, 2, \dots, C\}$
Parameters (what to learn)	$w \in \mathbb{R}^d$ (d is dimension of x)	$w \in \mathbb{R}^d$	$w^{(1)}, \dots, w^{(C)} \in \mathbb{R}^d$ (C*d total params)

Review: Supervised learning methods (so far)

	Linear Regression	Logistic Regression	Softmax Regression
Task	<u>Regression</u> y is a real number	<u>Binary Classification</u> $y \in \{+1, -1\}$	<u>Multiclass classification</u> $y \in \{1, 2, \dots, C\}$
Parameters (what to learn)	$w \in \mathbb{R}^d$ (d is dimension of x)	$w \in \mathbb{R}^d$	$w^{(1)}, \dots, w^{(C)} \in \mathbb{R}^d$ (C*d total params)
Probabilistic Story (how did nature create the data?)	$y \sim \text{Normal}(w^T x, \sigma^2)$ Mean (Depends on w) → Variance (constant) ↗	$p(y = 1 \mid x) = \sigma(w^T x)$ Plot of $\sigma(z)$ vs. z 	$p(y = j \mid x) = \frac{\exp(w^{(j)T} x)}{\sum_{k=1}^C \exp(w^{(k)T} x)}$ Normalizes to probability distribution

Review: Supervised learning methods (so far)

	Linear Regression	Logistic Regression	Softmax Regression
Task	<u>Regression</u> y is a real number	<u>Binary Classification</u> $y \in \{+1, -1\}$	<u>Multiclass classification</u> $y \in \{1, 2, \dots, C\}$
Parameters (what to learn)	$w \in \mathbb{R}^d$ (d is dimension of x)	$w \in \mathbb{R}^d$	$w^{(1)}, \dots, w^{(C)} \in \mathbb{R}^d$ (C*d total params)
Probabilistic Story (how did nature create the data?)	$y \sim \text{Normal}(w^\top x, \sigma^2)$ Mean (Depends on w) → Variance (constant)	$p(y = 1 \mid x) = \sigma(w^\top x)$ Plot of $\sigma(z)$ vs. z 	$p(y = j \mid x) = \frac{\exp(w^{(j)\top} x)}{\sum_{k=1}^C \exp(w^{(k)\top} x)}$ Normalizes to probability distribution
Loss function (measures how bad any choice of parameters is)	Derive using <u>Principle of Maximum Likelihood Estimation (MLE)</u> Want to maximize probability of data = $\prod_{i=1}^n p(y^{(i)} \mid x^{(i)}; w)$ Same as minimizing negative log-likelihood = $\sum_{i=1}^n -\log p(y^{(i)} \mid x^{(i)}; w)$		

Review: Supervised learning methods (so far)

	Linear Regression	Logistic Regression	Softmax Regression
Task	<u>Regression</u> y is a real number	<u>Binary Classification</u> $y \in \{+1, -1\}$	<u>Multiclass classification</u> $y \in \{1, 2, \dots, C\}$
Parameters (what to learn)	$w \in \mathbb{R}^d$ (d is dimension of x)	$w \in \mathbb{R}^d$	$w^{(1)}, \dots, w^{(C)} \in \mathbb{R}^d$ (C*d total params)
Probabilistic Story (how did nature create the data?)	$y \sim \text{Normal}(w^T x, \sigma^2)$ Mean $\rightarrow$ (Depends on w) Variance $\rightarrow$ (constant)	$p(y = 1 x) = \sigma(w^T x)$ Plot of $\sigma(z)$ vs. z 	$p(y = j x) = \frac{\exp(w^{(j)T} x)}{\sum_{k=1}^C \exp(w^{(k)T} x)}$ Normalizes to probability distribution
Loss function (measures how bad any choice of parameters is)	Derive using <u>Principle of Maximum Likelihood Estimation (MLE)</u> Want to maximize probability of data = $\prod_{i=1}^n p(y^{(i)} x^{(i)}; w)$ Same as minimizing negative log-likelihood = $\sum_{i=1}^n -\log p(y^{(i)} x^{(i)}; w)$		
How to minimize loss	Gradient Descent or Normal Equations	Gradient Descent	

Today's Plan

- Generative vs. Discriminative Classifiers
- Naïve Bayes for Text Classification
 - First Attempt
 - Two fixes to avoid zeroes
- Naïve Bayes for Feature Vectors

Discriminative Classifiers

- Train a model with parameters w to model $p(y/x)$
 - Logistic regression: $p(y=1/x) = \sigma(w^T x)$
- Note: We **do not** attempt to model $p(x)$!
 - **Given** an image x , classifier predicts whether it is a bird or not
 - Model does not try to describe what an image of a bird actually is
 - Only has to find some features that **discriminate** between birds and non-birds
- Methods like logistic regression & softmax regression are called “**discriminative classifiers**”



Decision Boundary
 $w^T x = 0$

Today: Generative classifiers

- Instead of modeling $p(y/x)$, **model the entire joint distribution $p(x, y)$** as the product $p(y) * p(x/y)$
 - $p(y)$: How often does each label occur? **Easy**
 - $p(x/y)$: What is the space of all possible x 's that have the label y ? **Can be complex**

Prior: 25% of images are birds

If y =bird,
all possible x 's include...



If y =not bird,
all possible x 's include...



Predicting with a Generative Classifier

- Suppose we have adequately learned $p(y)$ and $p(x/y)$
- At test time, we get an input x
- How to predict? **Bayes Rule**

Prediction of
label given input

$$p(y | x) = \frac{p(y)p(x | y)}{p(x)}$$

Model estimates these

Just for normalization

$$p(x) = \sum_j p(y = j)p(x | y = j)$$

Prior: 25% of images are birds

If y =bird,
all possible x 's include...



Test input



Today's Plan

- Generative vs. Discriminative Classifiers
- Naïve Bayes for Text Classification
 - First Attempt
 - Two fixes to avoid zeroes
- Naïve Bayes for Feature Vectors

Setting: Text Classification

- Each input x is a document
 - Documents can have different numbers of words
 - $x^{(i)}_j$ is j -th word of i -th training example
- Each training example has corresponding label y

Training Data (sentiment analysis)

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

Test Data

x^{test} = "great directing"

Training a generative classifier

- We have to model two things
 - $p(y)$: For each label y , what is the probability of y occurring?
 - $p(x|y)$: For each label y , what corresponding x 's are likely to appear?

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

Modeling $p(y)$

- Modeling $p(y)$ is **easy**: Just count how often each y appears!
- Let C be the number of possible classes
- Our model learns model parameter $\pi_j = P(y=j)$ for each possible j
- Learning: $\pi_j = \text{count}(y=j)/n$
 - $\text{count}(y=j)$: how often $y=j$ in training data
 - n : number of training examples
 - Justification: Maximum likelihood estimate (same as HW0 coin flip problem)

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

In this dataset: $y \in \{+1, -1\}$ so **$C=2$**

5 training examples, so **$n=5$**

$y=+1$ occurs 3 times, so $\pi_1 = 3/5 = 0.6$

$y=-1$ occurs 2 times, so $\pi_{-1} = 2/5 = 0.4$

Training a generative classifier

- We have to model two things
 - $p(y)$: For each label y , what is the probability of y occurring?
 - $p(x|y)$: For each label y , what corresponding x 's are likely to appear?
 - This is **much harder** because x 's are usually very complex objects
 - Different generative classification methods do different things
 - Today: **Naïve Bayes method**

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

Modeling $p(x|y)$ with Naïve Bayes

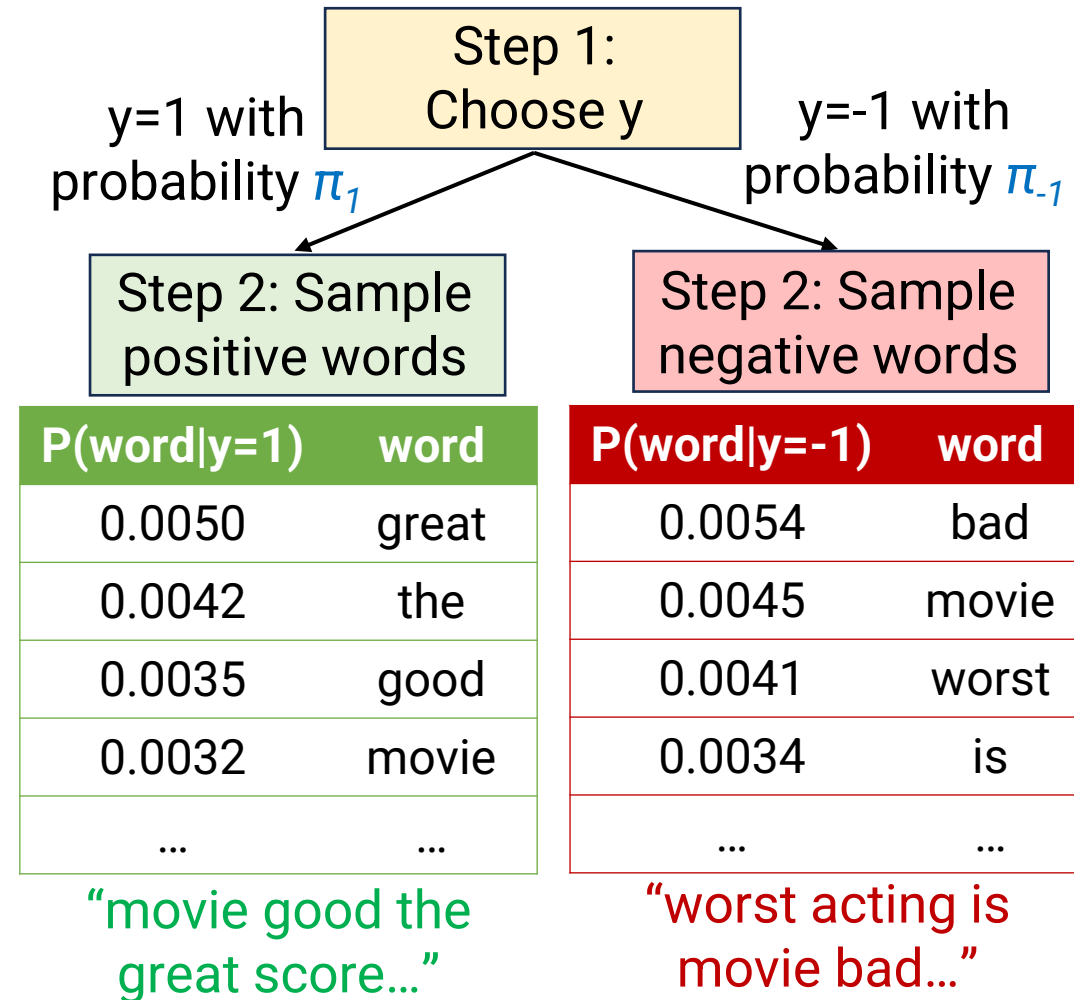
- Idea: Make a simplifying assumption about $p(x/y)$ to make it possible to estimate
- Naïve Bayes assumption: Each word of the document x is **conditionally independent** given label y :

$$p(x | y) = \prod_{j=1}^d p(x_j | y)$$

- “Once label is chosen, each word is sampled independently”
 - Note: This assumption does not have to be true (it definitely isn’t), just has to be “close enough” so that classifier makes reasonable predictions
- Note: This text classification model is called “*Multinomial Naïve bayes*” because each word is drawn from a multinomial distribution

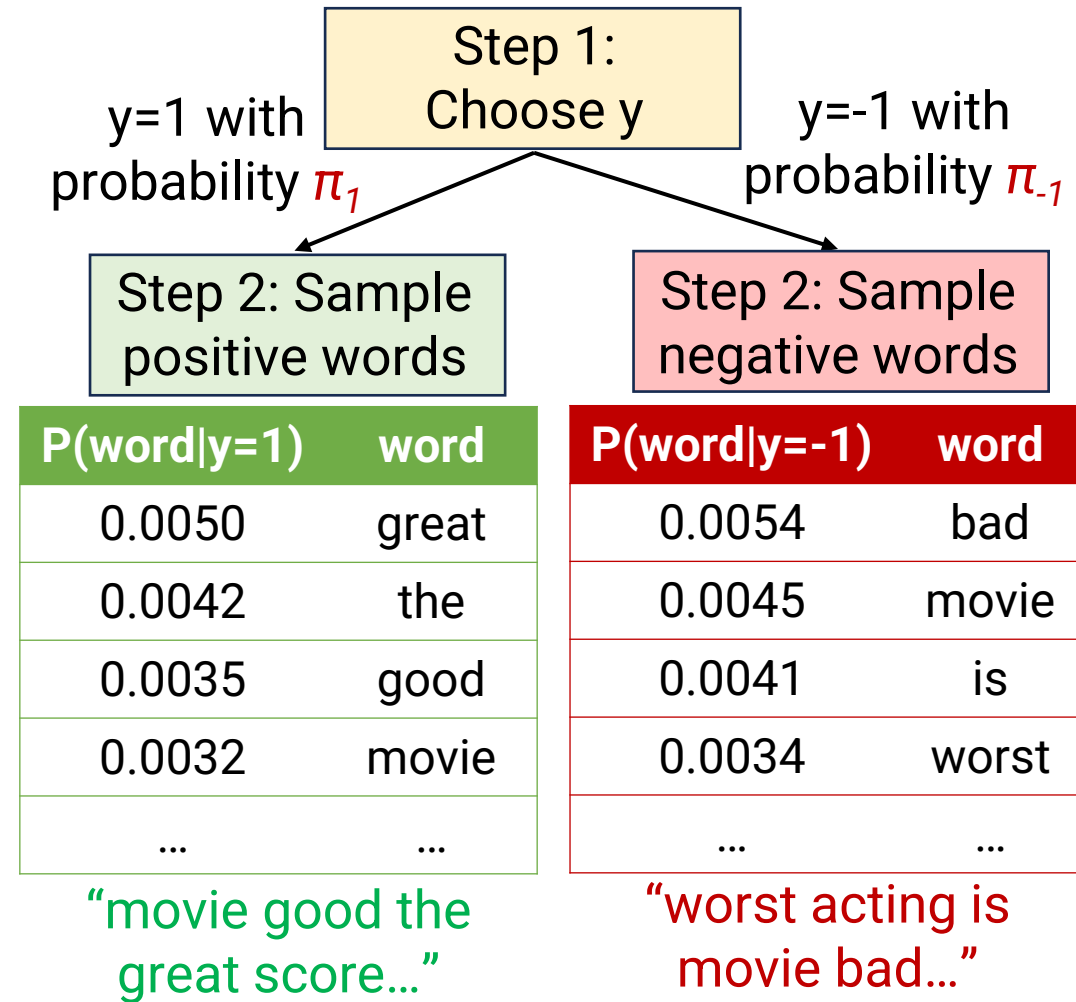
The Naïve Bayes Assumption

- Naïve Bayes posits its own **probabilistic story** about how the data was generated
- Step 1: Each $y^{(i)}$ was sampled from the prior distribution $p(y)$
 - “First, decide to either write a positive or negative review”
- Step 2: Each word in $x^{(i)}$ was sampled **independently** from the word distribution for label $y^{(i)}$
 - “If you decided to be positive, write the document by randomly sampling positive-sounding words”
 - “If you decided to be negative, write the document by randomly sampling negative-sounding words”
 - Each word is **independent when conditioning on y**
- Models the entire process of generating x and y



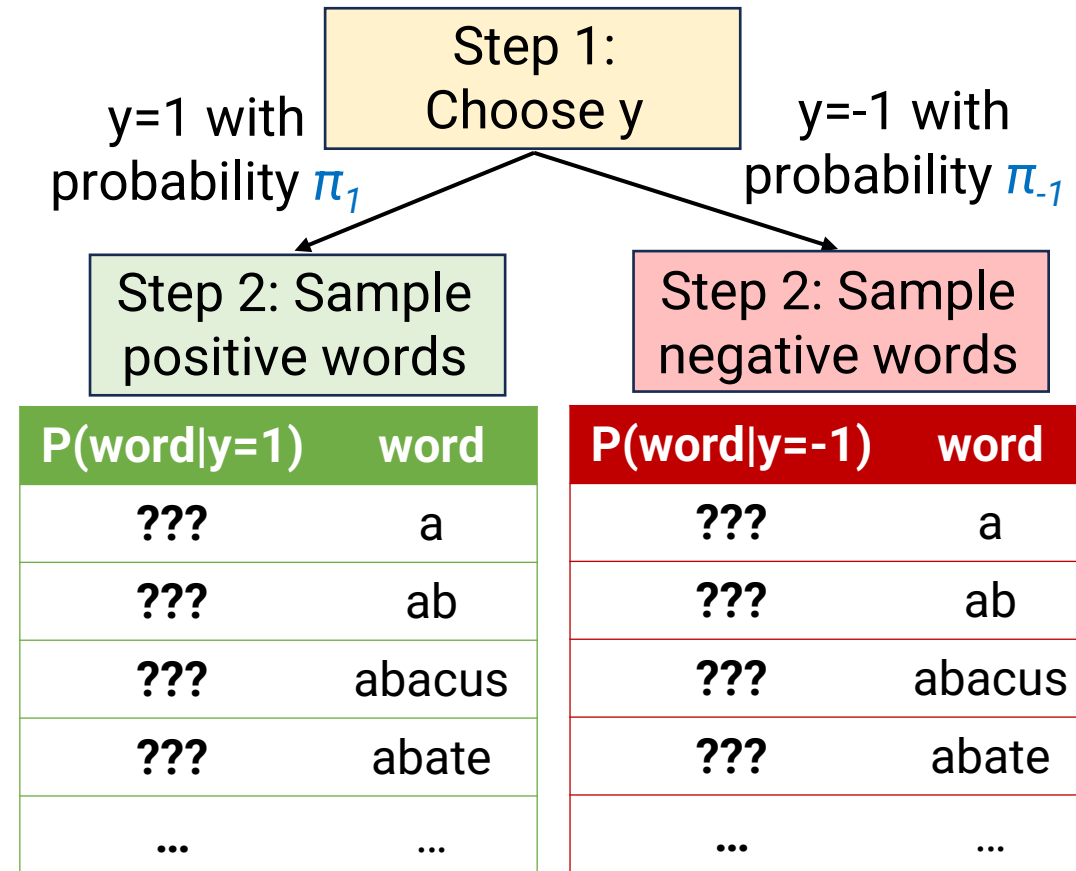
Why is the Naïve Bayes Assumption OK?

- Clearly, documents generated in this way don't look very realistic!
- Why is this OK?
 - We don't need our $p(x|y)$ to actually generate good documents
 - We just need it to be reasonable enough so that when given a real document x ,
$$p(x/\text{true } y) > p(x/\text{other } y)$$
 - Can be bad at modeling all the complex things that aren't related to y (grammar, writing style, etc.)



Learning with Naïve Bayes

- Let V (“vocabulary”) denote the set of words in the dictionary
- Model learns parameter $\tau_{wj} = P(w|y=j)$
 - For each word w in V
 - For each possible label j
 - Total of $|V| * C$ parameters to learn
- How to learn? Just count!
 - For each word w and label j , learn:
$$\tau_{wj} = \frac{[\text{\#occurrences of } w \text{ when } y=j]}{[\text{total words when } y=j]}$$
 - Again justified by MLE
 - **Note: This formula has a flaw, which we will fix later**



Learning goal: Estimate all the ???'s

Learning with Naïve Bayes

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

- For each of $y=+1$ and $y=-1$, want to learn a distribution over 8 words
- 7 total words appear with $y=+1$

Parameters to learn

$\tau_{w,1}$	word w	$\tau_{w,-1}$	word w
???	acting	???	acting
???	and	???	and
???	amazing	???	amazing
???	directing	???	directing
???	great	???	great
???	movie	???	movie
???	score	???	score
???	terrible	???	terrible

Learning with Naïve Bayes

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

- For each of $y=+1$ and $y=-1$, want to learn a distribution over 8 words
- 7 total words appear with $y=+1$
- Count each word and divide by total

Parameters to learn

$\tau_{w,1}$	word w	$\tau_{w,-1}$	word w
1/7	acting	???	acting
1/7	and	???	and
1/7	amazing	???	amazing
0	directing	???	directing
2/7	great	???	great
1/7	movie	???	movie
1/7	score	???	score
0	terrible	???	terrible

Learning with Naïve Bayes

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

- For each of $y=+1$ and $y=-1$, want to learn a distribution over 8 words
- 7 total words appear with $y=+1$
- Count each word and divide by total
- Repeat for $y=-1$ (3 total words)

Parameters to learn

$\tau_{w,1}$	word w	$\tau_{w,-1}$	word w
1/7	acting	0	acting
1/7	and	0	and
1/7	amazing	0	amazing
0	directing	1/3	directing
2/7	great	0	great
1/7	movie	0	movie
1/7	score	0	score
0	terrible	2/3	terrible

Predicting with Naïve Bayes

- Given test example $x^{\text{test}} = \text{"great score"}$
- Compute $p(x, y=+1)$
 - $= p(y=+1) * p(x|y=+1)$
 - $= p(y=+1) * p(\text{"great"}|y=+1) * p(\text{"score"}|y=+1)$
 - $= 3/5 * 2/7 * 1/7 = \mathbf{0.0245}$
- Compute $p(x, y=-1)$
 - $= p(y=-1) * p(x|y=-1)$
 - $= p(y=-1) * p(\text{"great"}|y=-1) * p(\text{"score"}|y=-1)$
 - $= 2/5 * 0 * 0 = \mathbf{0}$
- By Bayes Rule:
 - $P(y=+1|x) = 0.0245 / (0.0245 + 0) = 1$
 - Model is sure that $y=+1$, so predict +1
 - Always predict y with largest $p(x, y)$

Learned Parameters

$$\pi_1 = 3/5$$

$\tau_{w,1}$	word w
1/7	acting
1/7	and
1/7	amazing
0	directing
2/7	great
1/7	movie
1/7	score
0	terrible

$$\pi_{-1} = 2/5$$

$\tau_{w,-1}$	word w
0	acting
0	and
0	amazing
1/3	directing
0	great
0	movie
0	score
2/3	terrible

Announcements

- HW1 out, due next Wednesday (9/23)
 - CP homework help sessions this week
- Project Proposal will be due Monday October 5
 - Will discuss more next Monday
 - Groups of up to 3
- Oral Exam Logistics (Week of 9/28)
 - Question format posted on Piazza
 - Material covered is up through 9/21 lecture
 - Sign-up sheet released later this week

Oral Exam Format

Alice built a machine learning model for [TASK]. She trained several different models on her training set and evaluated them on her test set. She found that the best model achieved [METRIC] on the test set. That model had [METRIC] on the training set.

Answer the following questions:

1. Is this regression, binary classification, or multi-class classification? How do you know?
2. Out of linear regression, logistic regression, and softmax regression, which would you choose and why?
3. The linear methods we covered in class often involve x , y , and w . Would [QUANTITY] be considered part of x , y , w , or none of the above?
4. What was missing from Alice's experimental procedure? Why is this important?
5. To what extent did Alice's model overfit the dataset? To what extent did it underfit? How do you know?
6. Give two recommendations for things Alice could do to improve the performance of her model. Explain why you chose these two recommendations.

Today's Plan

- Generative vs. Discriminative Classifiers
- Naïve Bayes for Text Classification
 - First Attempt
 - Two fixes to avoid zeroes
- Naïve Bayes for Feature Vectors

Problem #1: Too Many Zeroes

- Given test example $x^{\text{test}} = \text{"great directing"}$
- Compute $p(x, y=+1)$
 - $= p(y=+1) * p(x|y=+1)$
 - $= p(y=+1) * p(\text{"great"}|y=+1) * p(\text{"directing"}|y=+1)$
 - $= 3/5 * 2/7 * \mathbf{0} = \mathbf{0}$
- Compute $p(x, y=-1)$
 - $= p(y=-1) * p(x|y=-1)$
 - $= p(y=-1) * p(\text{"great"}|y=-1) * p(\text{"directing"}|y=-1)$
 - $= 2/5 * \mathbf{0} * 1/3 = \mathbf{0}$
- By Bayes Rule:
 - $P(y=+1|x) = \mathbf{0}/(\mathbf{0}+\mathbf{0}) = \mathbf{NaN}$
 - Model thinks this x^{test} is impossible!**

Learned Parameters

$$\pi_1 = 3/5$$

$\tau_{w,1}$	word w
1/7	acting
1/7	and
1/7	amazing
0	directing
2/7	great
1/7	movie
1/7	score
0	terrible

$$\pi_{-1} = 2/5$$

$\tau_{w,-1}$	word w
0	acting
0	and
0	amazing
1/3	directing
0	great
0	movie
0	score
2/3	terrible

Avoiding Zeroes with Laplace Smoothing

- Problem : Assign probability of 0 to many (word, label) pairs
- Solution: **Laplace Smoothing**
 - Imagine that every (word, label) pair was seen an additional λ times
 - λ is a new hyperparameter
 - New formula:

$$\tau_{wy} = \frac{[\text{\#occurrences of } w \text{ when } y=j] + \lambda}{[\text{total words when } y=j] + |V| * \lambda}$$

Add λ for each word in V ,
so total # of imaginary counts is $|V| * \lambda$

Parameters to learn

$\tau_{w,1}$	word w
1/7	acting
1/7	and
1/7	amazing
0	directing
2/7	great
1/7	movie
1/7	score
0	terrible

$\tau_{w,-1}$	word w
0	acting
0	and
0	amazing
1/3	directing
0	great
0	movie
0	score
2/3	terrible

Laplace Smoothing Example

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

Parameters to learn

$\tau_{w,1}$	word w	$\tau_{w,-1}$	word w
1/7	acting	0	acting
1/7	and	0	and
1/7	amazing	0	amazing
0	directing	1/3	directing
2/7	great	0	great
1/7	movie	0	movie
1/7	score	0	score
0	terrible	2/3	terrible

With no Laplace Smoothing

Laplace Smoothing Example

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

$$\tau_{wy} = \frac{[\text{\#occurrences of } w \text{ when } y=j] + \lambda}{[\text{total words when } y=j] + |V| * \lambda}$$

Parameters to learn

$\tau_{w,1}$	word w	$\tau_{w,-1}$	word w
$(1+1)/(7+8)$	acting	$(0+1)/(3+8)$	acting
$(1+1)/(7+8)$	and	$(0+1)/(3+8)$	and
$(1+1)/(7+8)$	amazing	$(0+1)/(3+8)$	amazing
$(0+1)/(7+8)$	directing	$(1+1)/(3+8)$	directing
$(2+1)/(7+8)$	great	$(0+1)/(3+8)$	great
$(1+1)/(7+8)$	movie	$(0+1)/(3+8)$	movie
$(1+1)/(7+8)$	score	$(0+1)/(3+8)$	score
$(0+1)/(7+8)$	terrible	$(2+1)/(3+8)$	terrible

Laplace Smoothing with $\lambda = 1$

Laplace Smoothing Example

Training Data

i	$y^{(i)}$	$x^{(i)}$
1	+1	great acting and score
2	-1	terrible directing
3	+1	great movie
4	-1	terrible
5	+1	amazing

$$\tau_{wy} = \frac{[\text{\#occurrences of } w \text{ when } y=j] + \lambda}{[\text{total words when } y=j] + |V| * \lambda}$$

Parameters to learn

$\tau_{w,1}$	word w	$\tau_{w,-1}$	word w
2/15	acting	1/11	acting
2/15	and	1/11	and
2/15	amazing	1/11	amazing
1/15	directing	2/11	directing
3/15	great	1/11	great
2/15	movie	1/11	movie
2/15	score	1/11	score
1/15	terrible	3/11	terrible

Laplace Smoothing with $\lambda = 1$

Laplace Smoothing Avoids Zeroes

- Given test example $x^{\text{test}} = \text{"great directing"}$
- Compute $p(x, y=+1)$
 - $= p(y=+1) * p(x|y=+1)$
 - $= p(y=+1) * p(\text{"great"}|y=+1) * p(\text{"directing"}|y=+1)$
 - $= 3/5 * 3/15 * 1/15 = 0.0080$
- Compute $p(x, y=-1)$
 - $= p(y=-1) * p(x|y=-1)$
 - $= p(y=-1) * p(\text{"great"}|y=-1) * p(\text{"directing"}|y=-1)$
 - $= 2/5 * 1/11 * 2/11 = 0.0066$
- By Bayes Rule:
 - $P(y=+1|x) = 0.0080 / (0.0080 + 0.0066) = .595$
 - Model thinks $y=+1$ is more likely

Learned Parameters

$$\pi_1 = 3/5$$

$$\pi_{-1} = 2/5$$

$\tau_{w,1}$	word w
2/15	acting
2/15	and
2/15	amazing
1/15	directing
3/15	great
2/15	movie
2/15	score
1/15	terrible

$\tau_{w,-1}$	word w
1/11	acting
1/11	and
1/11	amazing
2/11	directing
1/11	great
1/11	movie
1/11	score
3/11	terrible

Laplace Smoothing with $\lambda = 1$

Problem #2: Numerical Underflow

- Given **long** test example x^{test} = “great directing and acting, amazing score, ...”
- Compute $p(x, y=+1)$:
 - $= p(y=+1) * p(x|y=+1)$
 - $= p(y=+1) * p(\text{“great”}|y=+1) * p(\text{“directing”}|y=+1) * p(\text{“and”}|y=+1) * p(\text{“acting”}|y=+1) * \dots$
- If you actually try to do this on a computer, you will get 0!
 - Multiplying many small numbers results in **numerical underflow**
 - Result is so small that it becomes 0

Learned Parameters

$$\pi_1 = 3/5$$

$\tau_{w,1}$	word w
2/15	acting
2/15	and
2/15	amazing
1/15	directing
3/15	great
2/15	movie
2/15	score
1/15	terrible

$$\pi_{-1} = 2/5$$

$\tau_{w,-1}$	word w
1/11	acting
1/11	and
1/11	amazing
2/11	directing
1/11	great
1/11	movie
1/11	score
3/11	terrible

Laplace Smoothing with $\lambda = 1$

Use Log Space to Avoid Underflow

- Given long test example $x^{\text{test}} = \text{"great directing and acting, amazing score, ..."}$
- Instead compute $\log p(x, y=+1)$:
 - $= \log p(y=+1) + \log p(x|y=+1)$
 - $= \log p(y=+1) + \log p(\text{"great"}|y=+1) + \log p(\text{"directing"}|y=+1) + \log p(\text{"and"}|y=+1) + \log p(\text{"acting"}|y=+1) + \dots$
 - This will not underflow, just adding together some negative numbers
- At test time: compute $\log p(x, y=j)$ for each j , choose the j with largest value

Learned Parameters

$$\pi_1 = 3/5$$

$\tau_{w,1}$	word w
2/15	acting
2/15	and
2/15	amazing
1/15	directing
3/15	great
2/15	movie
2/15	score
1/15	terrible

$$\pi_{-1} = 2/5$$

$\tau_{w,-1}$	word w
1/11	acting
1/11	and
1/11	amazing
2/11	directing
1/11	great
1/11	movie
1/11	score
3/11	terrible

Laplace Smoothing with $\lambda = 1$

Today's Plan

- Generative vs. Discriminative Classifiers
- Naïve Bayes for Text Classification
 - First Attempt
 - Two fixes to avoid zeroes
- Naïve Bayes for Feature Vectors

Naïve Bayes for Feature Vectors

Text Classification Setting

- Each input x is a document
 - Documents can have different numbers of words
 - $x^{(i)}_j$ is j -th word of i -th training example
 - We made an implicit assumption that **position of words does not matter**—same distribution for 1st word of document, 2nd word, etc.

Feature Vector Setting

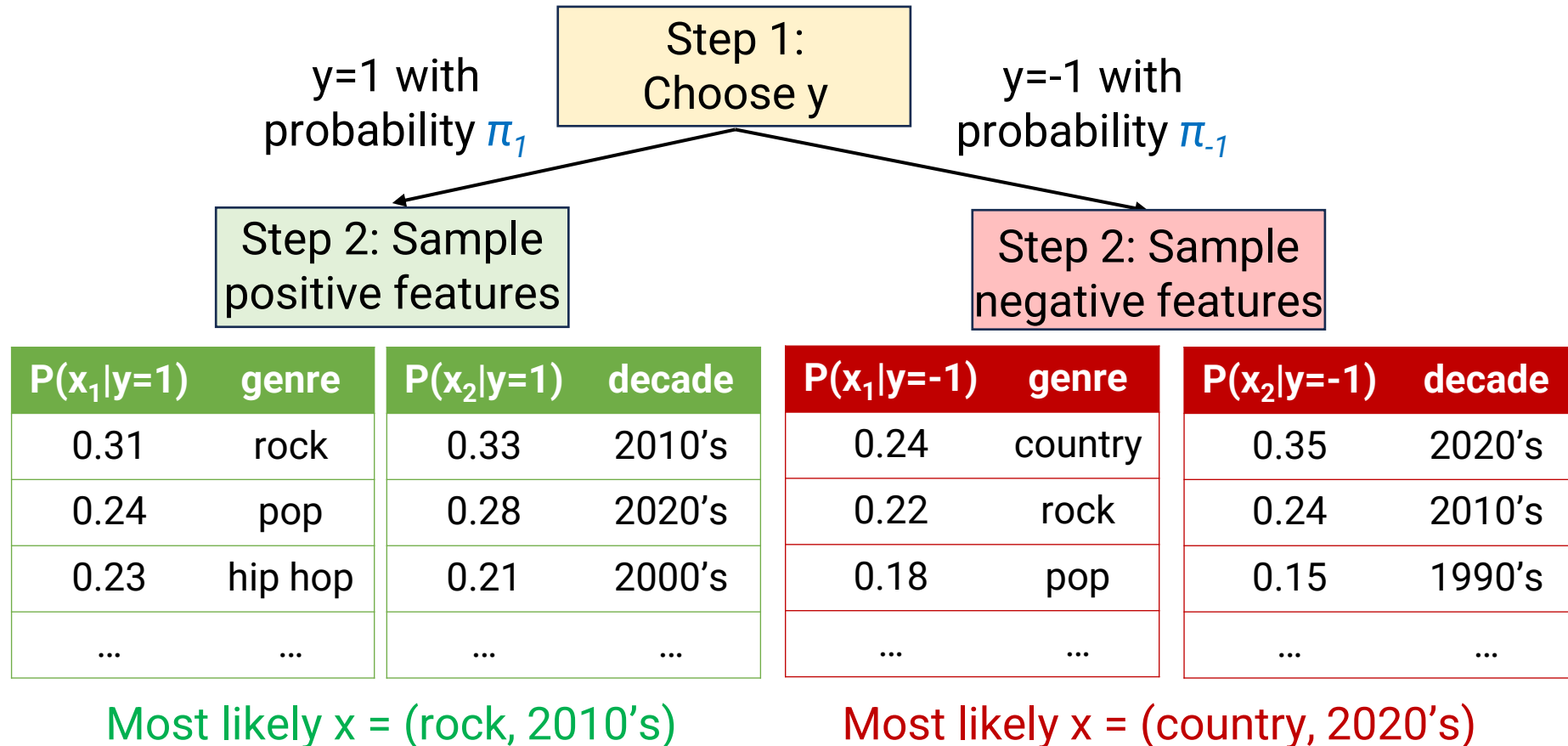
- Each input x is a feature vector
 - Each vector is of a fixed size d
 - $x^{(i)}_j$ is j -th feature of i -th training example
 - **Each feature means something different!** Can't treat them the same

Naïve Bayes for Feature Vectors

- Step 1: Each $y^{(i)}$ was sampled from the prior distribution $p(y)$
- Step 2: **For each** $j = 1, \dots, d$:

Feature $x_j^{(i)}$ was sampled independently from the **feature-specific** distribution for label $y^{(i)}$

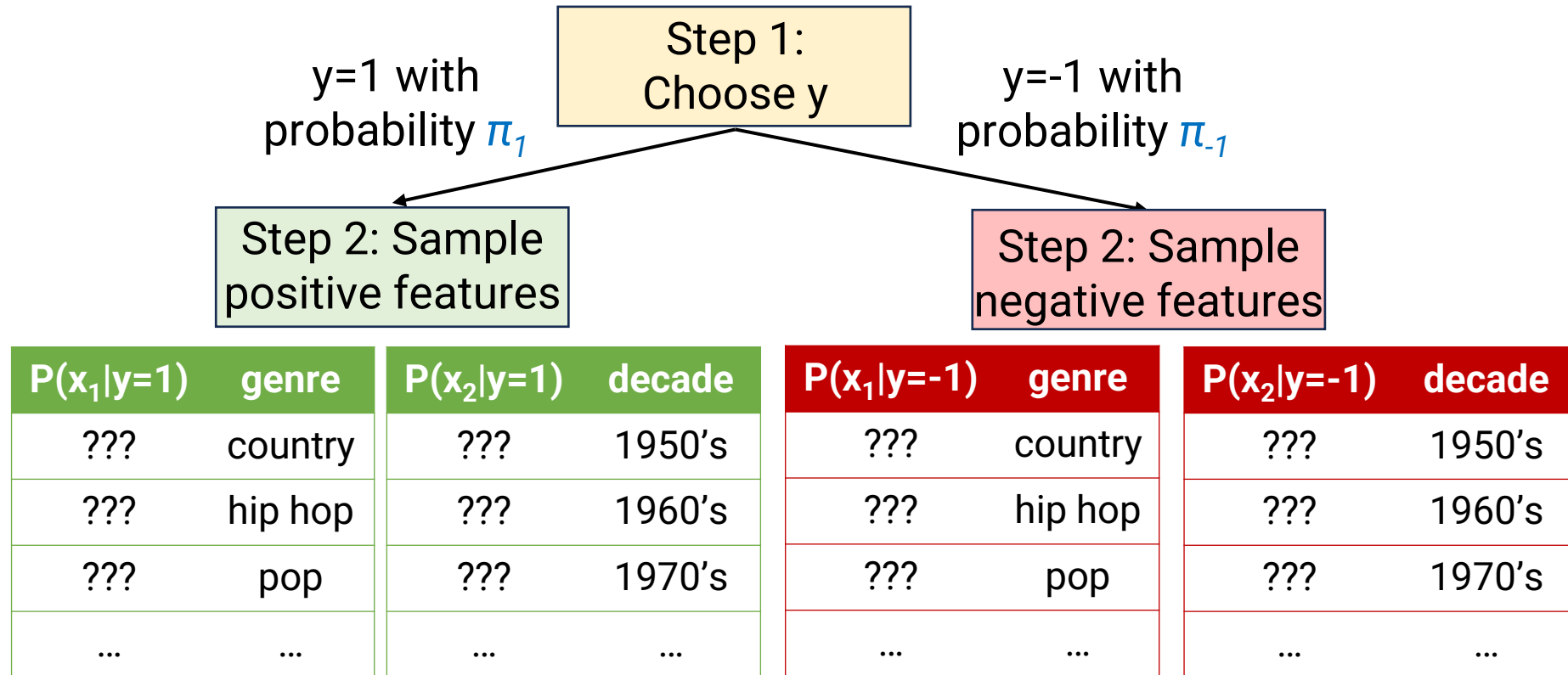
Task: Predict if user will like album (y) given genre (x_1) and decade (x_2)



Naïve Bayes for Feature Vectors

- How to learn?
Count occurrences for each feature
 - E.g., Count how many “liked” albums come from each genre
- Apply Laplace Smoothing to all (label, feature) pairs
 - E.g., Imagine 1 additional album of each genre was liked

Task: Predict if user will like album (y) given genre (x_1) and decade (x_2)



Discriminative vs. Generative Comparison

Logistic/Softmax Regression

- Usually higher accuracy, especially with large dataset
 - $P(y|x)$ usually simpler to learn than $P(x|y)$
- Can do arbitrary feature processing. Input features can be related to each other, since we don't make any conditional independence assumptions

Naïve Bayes

- Learning is easier—no gradient descent, just count!
- Can handle missing input features—just ignore them when computing $P(x|y)$
- Easy to make small updates to the model
 - New training example? Just increment counts
 - New label? Fit $P(x|y=\text{new label})$, everything else stays the same

Summary: Generative Classifiers, Naïve Bayes

- Generative Classifier: Model $p(y)$ and $p(x|y)$
 - Modeling $p(y)$ is **easy** (just count how often each label occurs)
 - Modeling $p(x|y)$ is **hard**
 - Naïve Bayes assumption: Each word/feature of x is **conditionally independent** given y
 - This makes modeling $p(x|y)$ easy: Just count!
 - Need to be careful to avoid zeroes
 - Laplace Smoothing to avoid zero probability of unseen (word, label) pairs
 - Work in log space to avoid numerical underflow
 - Use Bayes Rule to compute prediction $p(y|x)$ from $p(y)$ and $p(x|y)$